Natural Language Processing (NLP)

- **Learning Objectives**

  - Understand what NLP is and why it's important
  - Identify real-life applications of NLP
  - Know what will be covered in the course

Outline

1. What is NLP?
2. NLP tasks
3. How NLP works
4. Libraries for NLP
5. History of NLP
6. Applications of NLP
7. Course overview

What is NLP?

Natural Language Processing (NLP) is a branch of Artificial Intelligence that focuses on the interaction between computers and human languages. It combines computer science and linguistics to process and analyze large amounts of natural language data.

Why NLP?

With the explosion of digital data - social media, emails, reviews, etc. - there's a growing need to automatically understand and process human language. NLP helps in making sense of this data efficiently.

NLP Tasks

- Text Processing and Pre-processing
- Syntax and Parsing
- Semantic Analysis
- Information Extraction
- Text Classification in NLP
- Language Generation
- Speech Processing
- Question Answering
- Sentiment and Emotion Analysis

NLP Pre-processing Techniques

- **Tokenization:** Splitting text into smaller units like words or sentences.
- **Lowercasing:** Converting all text to lowercase to ensure uniformity.
- **Stop word Removal:** Removing common words that do not contribute significant meaning, such as "and," "the," "is."
- **Punctuation Removal:** Removing punctuation marks.
- **Stemming and Lemmatization:** Reducing words to their base or root forms. Stemming cuts off suffixes, while lemmatization considers the context and converts words to their meaningful base form.
- **Text Normalization:** Standardizing text format, including correcting spelling errors, expanding contractions and handling special characters.

How NLP works

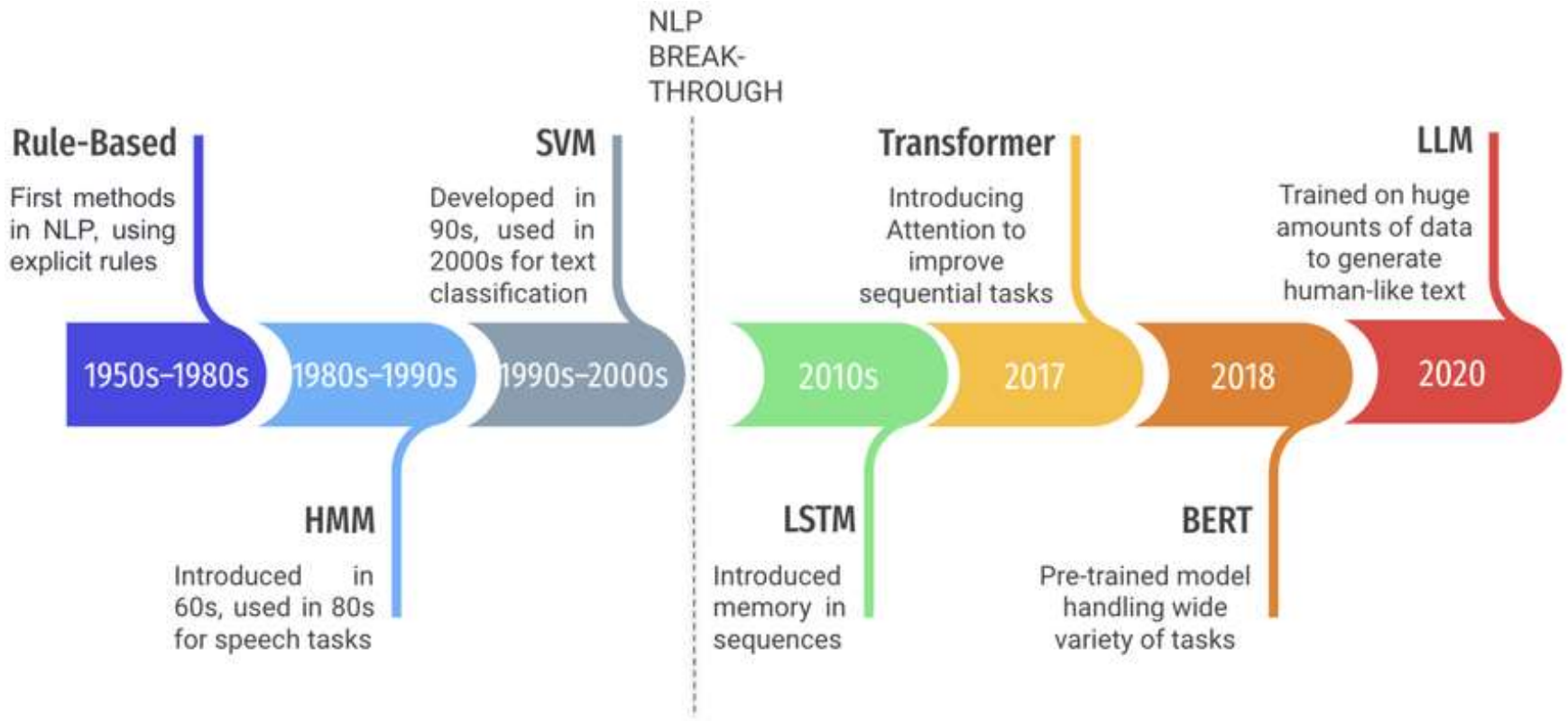
1. **Text Input and Data Collection**
2. **Text Pre-processing**
3. **Text Representation** (BoW, TF-IDF, Word Embeddings)
4. **Feature Extraction** (N-grams, Syntactic and Semantic)
5. **Model Selection and Training**
6. **Model Deployment and Inference**
7. **Evaluation and Optimization**

Libraries for NLP

Some of natural language processing libraries include:

1. NLTK (Natural Language Toolkit)
2. spaCy
3. TextBlob
4. Transformers (by Hugging Face)
5. Gensim
6. NLP Libraries in Python.

History of NLP



Applications of NLP

Course

DS743PE: Natural Language Processing
B.Tech IV Year I Sem (CSE - Data Science)
University: JNTU Hyderabad

- **Course Objectives:**
  - Understand key problems and solutions in NLP.
  - Explore the relation of NLP with linguistics and statistics.
  - Learn both rule-based and statistical models in NLP.

- **UNIT – I: Finding the Structure of Words & Documents**

  Key Topics:

  - **Morphology**: Study of word formation – roots, prefixes, suffixes
  - **Challenges in Morphological Analysis**: Dealing with new or compound words.
  - **Document Structure**: Understanding how documents are organized.

- **UNIT – II: Syntax I**

  Key Topics:

  - **Parsing Natural Language**: Breaking a sentence into its grammatical components.
  - **Treebanks**: Databases of parsed sentences used for training models.
  - **Parsing Algorithms**: Used to automatically generate syntactic structures.

- **UNIT – III: Syntax II & Semantic Parsing I:**
  - **Ambiguity Resolution**: How to resolve such ambiguity.
  - **Multilingual Issues**: Challenges of parsing in languages other than English.
  - **Semantic Parsing I**: Mapping sentences to their meanings.
  - **Word Sense Disambiguation**: Finding correct meaning for words with multiple senses.

- **UNIT – IV: Semantic Parsing II**
  - **Predicate-Argument Structure**: Who did what to whom? (e.g., *Ram* ate *mango*).
  - **Meaning Representation Systems**: Formal ways to represent meaning computationally.

- **UNIT – V: Language Modeling**

Key Topics:

- **N-gram Models**: Predicting next words based on previous ones.
- **Model Evaluation & Bayesian Methods**
- **Language Model Types**: Class-based, variable length, topic-based, etc.
- **Multilingual and Cross-lingual Models**

Key Terms in NLP

1. **Phonology**

   Phonology is the study of the **sound systems of languages**, how sounds are organized, patterned, and used.

2. **Morphology** helps understand how words are **constructed**, **modified**, and **related** to each other.

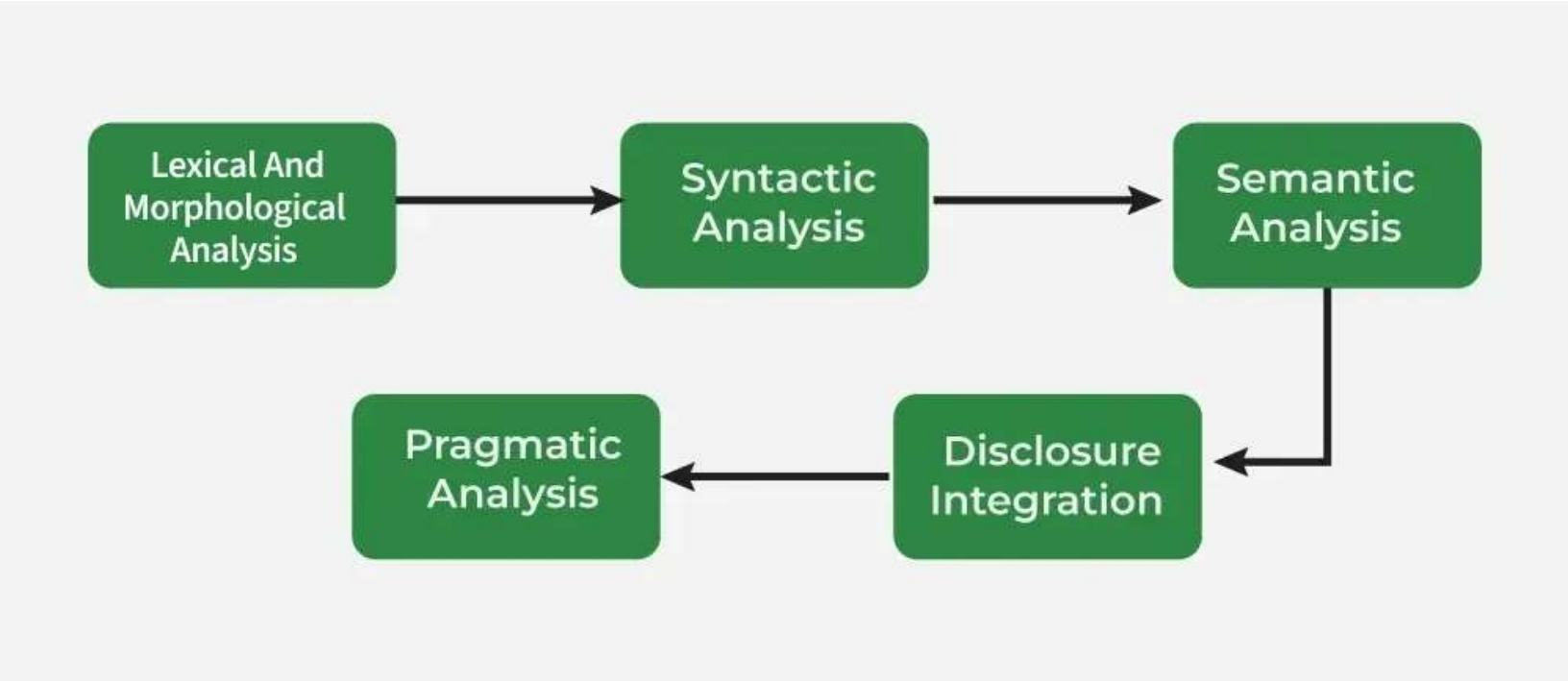
3. **Morpheme** is the **smallest meaningful unit of a word**. It cannot be divided further without losing or altering its meaning.

4. **Syntax** is the study of the **structure of sentences**—how words are arranged to form grammatically correct and meaningful phrases and sentences.

5. **Pragmatics** is the study of how **language is used in context**. It focuses on how people understand and produce language in **real-world situations**, including **tone, intent and assumptions**.

6. **Discourse** refers to how sentences are **connected** to form meaningful conversations, texts, or documents.

Phases of NLP



Phases of NLP

- **Lexical and Morphological Analysis**

  It focuses on identifying and processing words (or lexemes) in a text.

  Key tasks:

  Tokenization

  Parts- of- Speech tagging

- **Morphological Analysis**

  It deals with **morphemes** which are the smallest units of meaning in a word. It is important for understanding structure of words

  Key tasks:

  Stemming

  Lemmatization.

Phases of NLP

- **Syntactic Analysis (Parsing)**

  Syntactic Analysis helps in understanding how words in a sentence are arranged according to grammar rules.

  Key tasks:

  POS Tagging

  Ambiguity Resolution

- **Semantic Analysis**

  Semantic Analysis focuses on understanding meaning behind words and sentences. It ensures that the text is not only grammatically correct but also logically coherent and contextually relevant.**Key Key tasks:**

  Named Entity Recognition

  Word Sense Disambiguation

Phases of NLP

- **Discourse Integration**

  This phase ensures that the meaning of a text is consistent and coherent across multiple sentences or paragraphs.

  Key tasks:

  Anaphora Resolution

  Contextual References

- **Pragmatic Analysis**

  Pragmatic analysis helps in understanding the deeper meaning behind words and sentences by looking beyond their literal meanings.

  Key task:

  Understanding Intentions

Tokenization

Tokenization is a fundamental process in Natural Language Processing (NLP) that involves breaking down a stream of text into smaller units called tokens. Here are some types of tokenization,

1. **Word Tokenization**
2. **Character Tokenization**
3. **Sub-word Tokenization**
4. **Sentence Tokenization**
5. **N-gram Tokenization**.

Word tokenization

- **Word tokenization** is the process of **splitting a sentence or text into individual words**, which are called **tokens**.
- Example:

  Input sentence: "NLP is fun and useful!“

  After word tokenization: ['NLP', 'is', 'fun', 'and', 'useful', '!']
- Word tokenization is language-dependent

  Example: "don't" may be split as ['do', "n't"] or kept as 'don't' depending on the tokenizer.

Tools for Tokenization in Python

- Using NLTK:

```
from nltk.tokenize import word_tokenize
text = "NLP is fun and useful!"
tokens = word_tokenize(text)
print(tokens)
```

- Using spaCy:

```
import spacy
nlp = spacy.load("en_core_web_sm")
doc = nlp("NLP is fun and useful!")
tokens = [token.text for token in doc]
print(tokens)
```

Character tokenization

- **Character tokenization** is the process of **splitting a text into individual characters** rather than words or subwords.
- Example:

  Input text:"hello“

  After character tokenization:['h', 'e', 'l', 'l', 'o']

- Python Example using list():

```
text = "NLP"
char_tokens = list(text)
print(char_tokens)  # Output: ['N', 'L', 'P']
```

Sub-word tokenization

- **Sub-word tokenization** splits text into units **smaller than words but larger than characters,** such as prefixes, suffixes, or meaningful parts of a word.
- Example:

  Input word: "unhappiness“

  Sub-word tokens:['un', 'happi', 'ness']
- Methods:

Method	Description	Tools / Models
Byte-Pair Encoding (BPE)	Merges frequent pairs of characters/subwords	GPT, RoBERTa
WordPiece	Greedy longest-match subword segmentation	BERT
Unigram Language Model	Probabilistic subword selection	XLNet, SentencePiece

- Tools:
  - Hugging Face
  - TokenizersSentencePiece (Google)
  - SubwordTextEncoder (TensorFlow Datasets)

- Python code:

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
tokens = tokenizer.tokenize("unhappiness")
print(tokens)
# Output: ['un', '##happi', '##ness']
```

Sentence Tokenization

- Sentence Tokenization (also called **sentence segmentation**) is the process of **splitting a paragraph or text into individual sentences**.
- Example:

  Input text: "NLP is exciting. It has many applications!“

  After sentence tokenization:["NLP is exciting.", "It has many applications!"]

- Python code:

```
from nltk.tokenize import sent_tokenize
text = "NLP is exciting. It has many applications!"
sentences = sent_tokenize(text)
print(sentences)
```

OUTPUT: ['NLP is exciting.', 'It has many applications!']

N-gram tokenization

- **N-gram tokenization** splits text into **overlapping sequences of N items** (words or characters). These items are called **n-grams**.

  When **N = 1** → **Unigram**

  When **N = 2** → **Bigram**

  When **N = 3** → **Trigram**, and so on.

- **Types of N-grams:**

  1. **Word N-grams** – based on word sequences
     "New York City" → Unigram: 'New', 'York', 'City'
  2. **Character N-grams** – based on character sequences
     "hello", 2-grams: 'he', 'el', 'll', 'lo'

- Python code:

```
from nltk import ngrams
text = "I love NLP".split()
bigrams = list(ngrams(text, 2))
print(bigrams)
```

Types of Stemmer

1. Porter's Stemmer

- This method proposed by Martin Porter in 1980
- It has a set of pre-defined rules that govern the dropping of these affixes. It must be noted that stemmers might not always result in semantically meaningful base words.
- **Example:** EED -> EE means “if the word has at least one vowel and consonant plus EED ending, change the ending to EE” as 'agreed' becomes 'agree'.

- from nltk.stem import PorterStemmer

```
porter_stemmer = PorterStemmer()
words = "running", "runs", "easily", "fairly"

# Apply stemming to each word
stemmed_words = porter_stemmer.stem(word) for word in words
print("Original words:", words)
print("Stemmed words:", stemmed_words)
```

Output: 'run', 'run', 'easili', 'fairli'

2. Lovins Stemmer

- It is proposed by Lovins in 1968
- The longest suffix is removed from a word, and then the remaining stem is adjusted or modified to form a valid word.
- **Example:** running -> runn -> run

```
from stemming.porter2 import stem
words = "caresses", "flies", "flying", "happily", "running"
print("Lovins-like Porter2 Stemmer:")
for word in words:
    print(f"{word} → {stem(word)}")
```

3.Dawson Stemmer

- This method builds upon the original Lovins stemmer.
- In this version, suffixes (like "ing", "ation", "ness") are stored backward (e.g., "ing" becomes "gni", "ation" becomes "noita").
- The stemmer indexes or organizes these reversed suffixes based on:
  - Their length (number of characters)
  - Their last character (which is now the first character after reversal)
- This indexing structure helps the stemmer quickly look up suffixes that match the end of a word and apply stemming rules more efficiently.

4. Krovetz Stemmer

- It was proposed in 1993 by Robert Krovetz.
- 1) Convert the plural form of a word to its singular form.
  2) Convert the past tense of a word to its present tense and remove the suffix 'ing'.
  Example: 'children' -> 'child'

Python code:

```
from pykrovetzstemmer import Stemmer
kstem = Stemmer()
words = ["running", "skies", "organization", "children", "wolves"]
print("Krovetz Stemmer:")
for word in words:
    print(f"{word} → {kstem.stem(word)}")
```

5. N-Gram Stemmer

- Breaking words into segments of length n and then applying statistical analysis to identify patterns.
- An **n-gram** is a sequence of *n consecutive characters* from a word, where similar words share a high number of common n-grams.
- **Example:** 'SONS' for n=2 becomes : *S,SO,ON,NS,S*

```
from collections import defaultdict
from itertools import combinations
def ngrams(word, n=3):
    return set([word[i:i+n] for i in range(len(word)-n+1)])

def ngram_similarity(word1, word2, n=3):
    n1 = ngrams(word1, n)
    n2 = ngrams(word2, n)
    return len(n1 & n2) / len(n1 | n2)

word1 = "running"
word2 = "runner"

sim = ngram_similarity(word1, word2)
print(f"3-gram similarity between '{word1}' and '{word2}': {sim:.2f}")
```

6. Snowball Stemmer

- A more advanced and improved version of the Porter stemmer.
- Supports multiple languages.
- Developed using the Snowball language.

```
from nltk.stem import SnowballStemmer
ss = SnowballStemmer("english")
words = ["caresses", "flies", "flying", "happily", "running"]
print("Snowball Stemmer:")
for word in words:
    print(f"{word} → {ss.stem(word)}")
```

7. Lancaster Stemmer

- Also known as Paice-Husk Stemmer
- Very aggressive stemming
- Often chops off too much of the word
- But they are not as efficient as Snowball Stemmers.

```
from nltk.stem import LancasterStemmer
stemmer = LancasterStemmer()

words_to_stem = ['running', 'jumped', 'happily', 'quickly', 'foxes']
stemmed_words = [stemmer.stem(word) for word in words_to_stem]

print("Original words:", words_to_stem)
print("Stemmed words:", stemmed_words)
```

Output:

Original words: ['running', 'jumped', 'happily', 'quickly', 'foxes']

Stemmed words: ['run', 'jump', 'happy', 'quick', 'fox']

8. Regexp Stemmer

- Simple manual rule-based stemmer using regular expressions.
- Good for specific domains and quick testing.
- It utilizes regular expressions to identify and remove suffixes from words.

```
from nltk.stem import RegexpStemmer
custom_rule = r'ing$'
regexp_stemmer = RegexpStemmer(custom_rule)
word = 'running'
stemmed_word = regexp_stemmer.stem(word)
print(f'Stemmed Word: {stemmed_word}')
```

Output: run

```
import re

def simple_regex_stemmer(word):
    return re.sub('(ing|ly|ed|es|s)$', '', word)

words = ["running", "happily", "walked", "flies", "cars"]

print("Regex-based Custom Stemmer:")
for word in words:
    print(f"{word} → {simple_regex_stemmer(word)}")
```

Lemmatization

- Lemmatization is the process of converting a word to its base form.
- The difference between stemming and lemmatization is, lemmatization considers the context and converts the word to its meaningful base form, whereas stemming just removes the last few characters, often leading to incorrect meanings and spelling errors.
- For example, lemmatization would correctly identify the base form of ‘caring’ to ‘care’, whereas, stemming would cutoff the ‘ing’ part and convert it to car.

‘Caring’ -> Lemmatization -> ‘Care’ ‘Caring’ -> Stemming -> ‘Car’

Techniques

- Rule-Based Lemmatization
- Dictionary-Based Lemmatization
- Statistical & Machine Learning Approaches
- Hybrid Approaches

Rule-Based Lemmatization

- Uses hand-crafted linguistic rules (e.g., remove "-ing", "-ed") along with POS tags to infer base forms like "running" → "run"
- Fast, transparent, and effective for regular patterns—but misses irregulars and edge cases.

Dictionary-Based Lemmatization

- Relies on lexicons (e.g., WordNet) to map inflected forms to canonical lemmas (e.g., “better” → “good”)
- High accuracy for covered words, but fails on out-of-vocabulary items.

Statistical & Machine Learning Approaches

- Include CRFs, HMMs, and neural sequence-to-sequence models that learn patterns from annotated corpora .
- Adaptable and context-aware (handling ambiguous forms like “saw”), but require labeled data and are computationally heavier.

Hybrid Approaches

- Combine rule-based/dict methods with statistical models.Example: Stanford CoreNLP applies rules for regular forms, supplemented by ML for irregulars
- Offer a good balance of accuracy and coverage.

Methods

1. Wordnet Lemmatizer
2. Wordnet Lemmatizer with appropriate POS tag
3. spaCy Lemmatization
4. TextBlob Lemmatizer
5. TextBlob Lemmatizer with appropriate POS tag
6. Pattern Lemmatizer
7. Stanford CoreNLP Lemmatization
8. Gensim Lemmatize
9. TreeTagger
10. Comparing NLTK, TextBlob, spaCy, Pattern and Stanford CoreNLP

WordNetLemmatizer

```
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()

lemmas = [
    lemmatizer.lemmatize('dogs'),              # 'dog'
    lemmatizer.lemmatize('churches'),          # 'church'
    lemmatizer.lemmatize('aardwolves'),        # 'aardwolf'
    lemmatizer.lemmatize('abaci'),             # 'abacus'
    lemmatizer.lemmatize('hardrock')           # 'hardrock' (no
change)
]
```

Wordnet Lemmatizer with appropriate POS tag

```
import nltk
from nltk.corpus import wordnet


def penn_to_wn(treebank_tag):
    if treebank_tag.startswith('J'): return wordnet.ADJ
    if treebank_tag.startswith('V'): return wordnet.VERB
    if treebank_tag.startswith('N'): return wordnet.NOUN
    if treebank_tag.startswith('R'): return wordnet.ADV
    return None
```

```
tokens = nltk.word_tokenize(text)
tagged = nltk.pos_tag(tokens)
lemmatized = [
    (word, lemmatizer.lemmatize(word, pos=pos) if (pos := penn_to_wn(tag)) else lemmatizer.lemmatize(word))
    for word, tag in tagged
]
```

spaCy Lemmatization

- It comes with pre-built models that can parse text and compute various NLP related features through one single function call.

```
# Install spaCy (run in terminal/prompt)
import sys
!{sys.executable} -m pip install spacy

# Download spaCy's  'en' Model
!{sys.executable} -m spacy download en
```

```
import spacy
# Load the spaCy English model
nlp = spacy.load('en_core_web_sm')
text = "The quick brown foxes are jumping over the lazy dogs.“

# Process the text using spaCy
doc = nlp(text)
lemmatized_tokens = [token.lemma_ for token in doc]
lemmatized_text = ' '.join(lemmatized_tokens)
print("Original Text:", text)
print("Lemmatized Text:", lemmatized_text)
```

Original Text: The quick brown foxes are jumping over the lazy dogs.

Lemmatized Text: the quick brown fox be jump over the lazy dog .

Code 2:

```
import spacy
# includes tokenizer, tagger, and lookup lemmatizer
nlp = spacy.load("en_core_web_sm")


# optional: use only relevant pipes for speed
with
nlp.select_pipes(enable=["tok2vec","tagger","attribute_ruler","le
mmatizer"]):
    doc = nlp("Cats chasing mice")
    print([token.text + " → " + token.lemma_ for token in doc])
    # Cats → cat, chasing → chase, mice → mouse
```

TextBlob Lemmatizer

- TexxtBlob is a powerful, fast and convenient NLP package as well.
- Using the Word and TextBlob objects, its quite straighforward to parse and lemmatize words and sentences respectively.

```
# pip install textblob
from textblob import TextBlob, Word
word = 'stripes'
w = Word(word)
w.lemmatize()
```

Output: stripe

```
# Lemmatize a sentence
sentence = "The striped bats are hanging on their feet for best"
sent = TextBlob(sentence)
" ". join([w.lemmatize() for w in sent.words])
```

Output: 'The striped bat are hanging on their foot for best'

TextBlob Lemmatizer with appropriate POS tag

```
def lemmatize_with_postag(sentence):
    sent = TextBlob(sentence)
    tag_dict = {"J": 'a',
                "N": 'n',
                "V": 'v',
                "R": 'r'}
words_and_tags = [(w, tag_dict.get(pos[0], 'n')) for w, pos in sent.tags]
lemmatized_list = [wd.lemmatize(tag) for wd, tag in words_and_tags]
    return " ".join(lemmatized_list)
# Lemmatize
sentence = "The striped bats are hanging on their feet for best"
lemmatize_with_postag(sentence)
```

Pattern's lemmatizer

- Pattern is a versatile Python library for NLP among other tasks, and includes a rule-based lemmatizer using built-in English lexica and suffix rules two key
- functions:
  - lemma(word): returns the base form.
  - lexeme(word): returns all inflectional variants

```
from pattern.en import lemma, lexeme


sentence = "the bats saw the cats with best stripes hanging upside down by their feet"
lemmas = [lemma(w) for w in sentence.split()]
print(" ".join(lemmas))
```

Output: "the bat see the cat with best stripe hang upside down by their feet"

Stanford CoreNLP's lemmatization

- Context-sensitive and POS-aware: leverages POS tags to determine accurate lemmas.
- Reliable for English and other well-supported languages.Integrated into CoreNLP's full linguistic pipeline (parsing, NER, dependency analysis, etc.).
- Offers both low-level (token) and high-level (sentence-level via Sentence.lemmas()) APIs.

- From the commandline

```
java -Xmx5g \
  edu.stanford.nlp.pipeline.StanfordCoreNLP \
  -annotators tokenize,ssplit,pos,lemma \
  -file input.txt
```

- From java code

```
Properties props = new Properties();
props.setProperty("annotators", "tokenize,ssplit,pos,lemma");
StanfordCoreNLP pipeline = new StanfordCoreNLP(props);
CoreDocument doc = pipeline.processToCoreDocument("Marie was born in Paris.");
for (CoreLabel tok : doc.tokens()) {
    System.out.println(tok.word() + "\t" + tok.lemma());
}
```

```
from stanfordcorenlp import StanfordCoreNLP
nlp = StanfordCoreNLP('http://localhost', port=9000)
props = {'annotators': 'pos,lemma', 'pipelineLanguage': 'en',
'outputFormat': 'json'}
res = nlp.annotate("The bats saw the cats.", properties=props)
```

Output: Each token in res['sentences'][0]['tokens'] includes a "lemma" field

Thank you